

Problems of Shared-Exclusive Locking Protocol

DBMS Concurrency Control — GATE / Competitive Exam Notes

GATE SMASHERS SERIES

SUBJECT: DBMS

TOPIC: LOCK-BASED PROTOCOLS

1. Advantage & Limitations Overview

✓ Key Advantage

Simple Method: Extremely straightforward locking mechanism, minimal programming overhead, and simple software/DBMS implementation.

🚨 4 Major Limitations

Basic Shared-Exclusive locking alone is insufficient to guarantee complete ACID consistency and transactional safety without additional constraints (e.g., 2PL).

🧠 MEMORY TRICK

"Simple Lock has 4 Problems: S-R-D-S"

S Serializability | **R** Recoverability | **D** Deadlock | **S** Starvation

2. Problem 1: May Not Guarantee Serializability

A very common exam trap is assuming that locking automatically guarantees a serializable schedule. Basic shared-exclusive locking allows locks to be released before all data accesses are completed, leading to interleaved non-serial executions.

Shared-Exclusive Locking $\nrightarrow$ Always Serializable Schedule

🔥 Interleaved Execution Example

Consider two transactions, T1 and T2, operating on items A and B:

T1: Lock-S(A) -> Read(A) -> Unlock(A) ↓ (Interrupted by T2) T2: Lock-X(A) -> Read(A) -> Write(A) -> Unlock(A) ↓ (T1 Resumes) T1: Lock-S(B) -> Read(B) -> Write(B) -> Unlock(B) Order of Execution: T1 → T2 → T1 (Non-serial conflict pattern)

★ Exam Statement Check

Statement: "Shared-exclusive locking may not always be sufficient to produce serializable schedules."

Answer: **TRUE** — Simple locking controls concurrent data access, but without growth/shrink rules (like 2PL), non-serializable interleaving can occur.

3. Problem 2: May Not Be Free from Irrecoverability

Shared-exclusive locking controls **READ** and **WRITE** operations using locks, but it does not specify or enforce **COMMIT** or **ROLLBACK** ordering relative to other transactions.

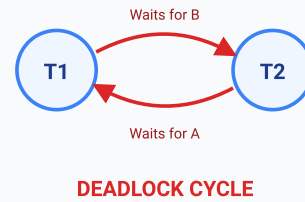
T1: Obtains X(A) -> Writes A -> Unlocks A T2: Obtains S(A) -> Reads A -> COMMITS T1: Encounters failure -> FAILS -> ROLLBACK (Needs to revert A)

Because **T2 committed** after reading dirty data produced by **T1** (which later rolled back), T2 cannot be rolled back. This creates an **Irrecoverable Schedule**.

4. Problem 3: Deadlock (Infinite Waiting)

Deadlock occurs when two or more transactions are waiting indefinitely for locks held by each other, creating a circular wait cycle in the Wait-For Graph (WFG).

T1 holds Exclusive Lock on A: X(A) T2 holds Exclusive Lock on B: X(B) T1 requests Lock on B → WAITS for T2 T2 requests Lock on A → WAITS for T1 Result: Cyclic Mutual Waiting



5. Problem 4: Starvation (Long/Indefinite Delay)

Starvation happens when a transaction is repeatedly delayed from obtaining a lock because a stream of other transactions continuously acquire compatible locks on the same data item.

T1 requests Exclusive Lock: X(A) → WAITS (T2 currently holds Shared Lock S(A)) T3 requests Shared Lock: S(A) → GRANTED (Compatible with T2's S(A)) T4 requests Shared Lock: S(A) → GRANTED (Compatible with T3's S(A)) ... T1 continues waiting indefinitely while new transactions get S(A) ...

6. Deadlock vs. Starvation Comparison

Feature	Deadlock	Starvation
Waiting Nature	Mutual / Cyclic waiting	One-sided / Delayed waiting
Duration	Infinite (cannot resolve without intervention)	Long / Finite (eventually may complete)
Cause	Circular dependency cycle in resources	Concurrency bias / Continuous shared locks
Resolution	Requires transaction Abort / Rollback	Requires Priority Queue / Fair Locking (FIFO)

7. Master Summary & Quick Exam Practice

Question / Statement	Ans	Core Reason / Explanation
Shared-exclusive locking always guarantees serializability.	FALSE	Unlocking early allows non-serial interleaving.
Shared-exclusive locking always guarantees recoverability.	FALSE	Does not enforce commit order relative to dirty reads.
Deadlock and Starvation are identical concurrency issues.	FALSE	Deadlock is infinite mutual wait; Starvation is starvation by priority.
Two-Phase Locking (2PL) is introduced to solve serializability.	TRUE	2PL adds growing and shrinking phases to ensure serializability.



One-Line Exam Takeaway

Shared-exclusive locking controls concurrent data access using S and X locks, but by itself, it does **NOT** guarantee serializability or recoverability, and it remains vulnerable to both **deadlock** and **starvation**.